COSC 39: Theory of Computation

Credit Statement: Talked to Carson Bates'27, and Kiran Jones'27.

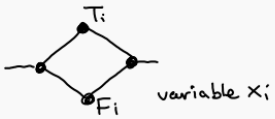
Problem 1. An induced cycle in an undirected simple graph G is a subgraph $G[S]$ induced on the vertices in S that happens to be a cycle. (Intuitively, an induced cycle is a cycle in G without chords.)

INDUCEDCYCLE

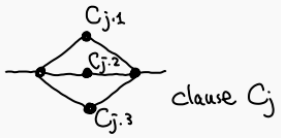
- **Input:** An undirected simple graph G, two vertices u and v
- **Output:** Is there an induced cycle in G passing through u and v?

1. Why doesn't the following construction work?

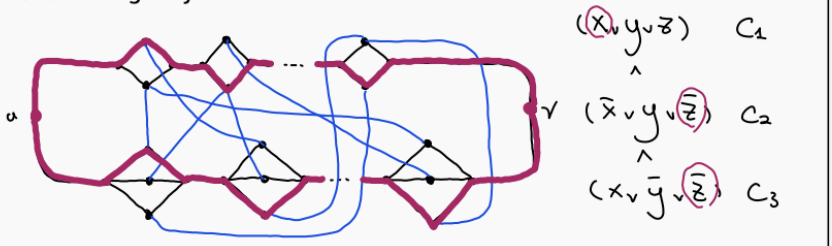
Given 3CNF formula ϕ, for each variable create a gadget:



For each clause create a gadget:



Add an edge between node f_i and node $c_{j,\ell}$ if x_i is the ℓ-th literal of clause C_j; add an edge between node t_i and node $c_{j,\ell}$ if $\overline{x_i}$ is the ℓ-th literal of clause C_j. Link the gadgets in the following way:



Now there is an induced cycle passing through u and v if and only if there are variable assignments for $\langle x_i \rangle$ that satisfy all clauses. $x = T,\ y = F,\ z = F$

2. From your diagnosis, if the input graph is allowed to be non-simple and have parallel edges, how would you resolve the issue?
3. Alas INDUCEDCYCLE only allows simple graphs. Still, prove that INDUCEDCYCLE is NP-hard. Proceed in the order of the following steps:

(a) Can you mimick the fix for the non-simple case with some kind of duplication?

(b) After the duplication, how do you ensure consistency?

(c) Why don't the edges added for consistency cause the exact same problem as before?

(d) This ties to the placement of the duplicated gadgets; how do you ensure the induced cycle passes through them? Observe that the cycle has to pass through u and v.

Solution.

1. The construction doesn't work because not every induced cycle corresponds to a satisfying assignment. For instance, there may exist an induced cycle that travels from u through a blue edge into a clause gadget, then continues to v, and then returns through another blue edge into a different clause gadget back to u. This cycle skips the intended variable-assignment structure, and hence does not encode a satisfying assignment.

2. If the graph were allowed to be non-simple, we could fix the issue by replacing every blue edge with two parallel edges. Then any cycle using such an edge would automatically contain a chord, since the second parallel edge forms a 2-cycle.

3. (a) We mimick the fix for the non-simple case by duplicating the variable gadgets and arranging them in a palindromic format. That is, if the variables are $x_1, \ldots, x_n$ we connect the gadgets as follows,

$$X_1 - X_2 - \cdots - X_n - X_n' - \cdots - X_2' - X_1'$$

   where X_i' is a copy of X_i with the same connections to the clause gadgets.

   (b) We ensure consistency between the two copies by adding edges t_{x_i}—$f_{x_i'}$ and f_{x_i}—$t_{x_i'}$. This ensures that any induced cycle would have to pick the same value for both gadgets corresponding to x_i because otherwise one of the blue edges would become a chord, contradicting that the cycle is induced.

   (c, d) Now we only need to ensure that the cycle doesn't use one of these new edges between corresponding variable gadgets to skip some variable gadgets entirely. There are two cases we must look at.

      i. x_i is not a free variable in the CNF.
      Then the induced cycle must pass through a clause gadget containing either x_i or $\overline{x_i}$. If the cycle attempts to use one of the new consistency edges while

making inconsistent choices between the two copies of the variable gadget, one of the clause edges becomes a chord: either t_{x_i} is adjacent to a vertex corresponding to $\overline{x_i}$, or f_{x_i} is adjacent to a vertex corresponding to x_i. Hence the resulting cycle is not induced.

ii. x_i is a free variable in the CNF.
Then our argument in (i) fails because the cycle could use the new consistency edges to skip portions of the construction without creating a chord.
To fix this, before constructing the graph we modify the CNF by adding for each variable we add the clause, $(x_i \vee x_i \vee \overline{x_i})$. Since this clause is always satisfiable it doesn't change the satisfibility of the CNF. However, it guarantees that every variable appears in at least one clause gadget solving our problem.

Hence we can reduce 3SAT to INDUCEDCYCLE in polynomial time, thus proving that INDUCEDCYCLE is NP-hard.